Introduction to lesson

- In this lesson, you will learn how to implement KF in Octave.
  - I recommend that you keep the KF equations nearby.

Step 1a: State prediction time update

$\hat{x}_k^- = A_{k-1}\hat{x}_{k-1}^+ + B_{k-1}u_{k-1}$.

Step 1b: Prediction-error covariance time update

$\Sigma_{\tilde{x},k}^- = A_{k-1}\Sigma_{\tilde{x},k-1}^+ A_{k-1}^T + \Sigma_{\tilde{w}}$.

Step 1c: Predict system output

$\hat{z}_k = C_k\hat{x}_k^- + D_k u_k$.

Prediction

Step 2a: Estimator gain matrix

$L_k = \Sigma_{\tilde{x},k}^- C_k^T [C_k \Sigma_{\tilde{x},k}^- C_k^T + \Sigma_{\tilde{v}}]^{-1}$.

Step 2b: State estimate measurement update

$\hat{x}_k^+ = \hat{x}_k^- + L_k(z_k - \hat{z}_k)$.

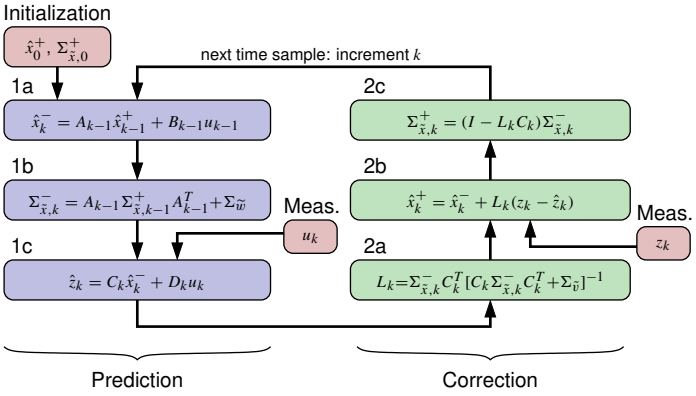
Step 2c: Estimation-error covariance measurement update

$\Sigma_{\tilde{x},k}^+ = (I - L_k C_k)\Sigma_{\tilde{x},k}^-$.

Correction

Visualizing the linear Kalman filter

- Linear Kalman-filter equations naturally form a recursion:



- "Simple" to implement on a digital computer.
- Only six lines of code execute each iteration!
- Most of the work is going to be setting up the variables and plotting results.

KF step 1 (prediction)

- Code loads sim data, then implements main KF loop, step 1:

```
% Load data from simulation of system dynamic model
load simOut.mat Ad Bd Cd Dd SigmaV SigmaW dT t u x z

% Determine number of states and timesteps
[nx,nt] = size(x);
% Initialize state estimate and covariance
xhat = zeros(nx,1); SigmaX = zeros(nx,nx);
% Initialize storage for state/bounds for plotting purposes
xhatstore = zeros(nx,nt); boundstore = zeros(nx,nt);

for k = 2:length(t)
  % KF Step 1a: State prediction time update
  xhat = Ad*xhat + Bd*u(:,k-1); % use prior value of "u"

  % KF Step 1b: Prediction-error covariance time update
  SigmaX = Ad*SigmaX*Ad' + SigmaW;

  % KF Step 1c: Predict system output
  zhat = Cd*xhat + Dd*u(:,k); % use present value of "u"
```

KF step 2 (correction)

- Code continues with main KF loop, step 2:

```
  % KF Step 2a: Compute estimator matrix
  L = SigmaX*Cd'/(Cd*SigmaX*Cd' + SigmaV);

  % KF Step 2b: State estimate measurement update
  xhat = xhat + L*(z(:,k) - zhat);

  % KF Step 2c: Estimation-error covariance measurement update
  SigmaX = (eye(nx)-L*Cd)*SigmaX;

  % [Store estimate and bounds for evaluation/plotting purposes]
  xhatstore(:,k) = xhat;
  boundstore(:,k) = 3*sqrt(diag(SigmaX));
end
```

- That's it! We're now ready to plot results.

Plot results from KF execution

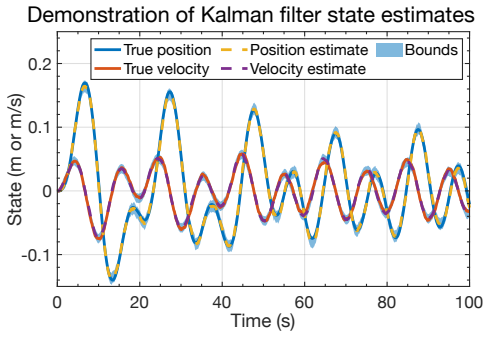
- Here is some code to plot results:

```
figure(1); clf;      % Plot the states and estimates
t2 = [t fliplr(t)]; % Prepare for plotting bounds via "fill"
x2 = [xhatstore-boundstore fliplr(xhatstore+boundstore)];
h1 = fill(t2,x2,'b','FaceAlpha',0.5,'LineStyle','none'); hold on; grid on
h2 = plot(t,x',t,xhatstore','--'); % Plot true state and its estimate
legend([h2;h1],'True position','True velocity','Position estimate',...
  'Velocity estimate','Bounds');
title('Demonstration of Kalman filter state estimates');
xlabel('Time (s)'); ylabel('State (m or m/s)');

figure(2); clf; xerr = x - xhatstore; % Plot state-estimation errors
for k = 1:nx
  subplot(nx,1,k); % Plot each state's error in a separate subplot
  fill(t2,[-boundstore(k,:) fliplr(boundstore(k,:))],'b',...
    'FaceAlpha',0.25,'LineStyle','none'); hold on; grid on;
  plot(t,xerr(k,:),'b');
  title(sprintf('State %d estimation error with bounds',k));
  xlabel('Time (s)'); ylabel('Error');
end
```

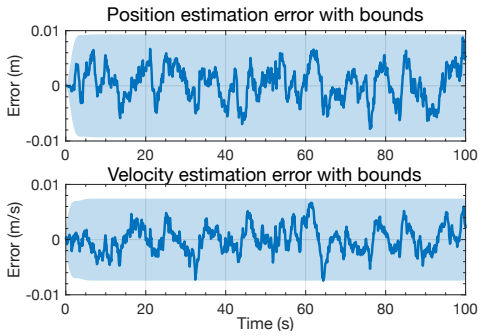

Sample results: States and estimates

- Plot shows output from the KF for the spring-mass-damper example, using simulation data from earlier this week.
- Here, we visualize the true states together with the estimated states, with shaded estimate confidence bounds.
- The KF does a good job of estimating both the position and velocity states in this example.
- The estimation errors never converge to zero, but the state estimates do appear to stay within the confidence bounds (both as expected).



Sample results: Estimation errors

- Plot shows an alternate way of viewing the output of the KF, looking directly at the state-estimation errors, $x_k - \hat{x}_k^+$.
- This type of plot makes it easier to see that state estimation errors don't converge to zero but always have a random component (due to w_k).
- It's also easier to see whether estimates always remain within confidence bounds:
  - Posn. estimate is always within bounds.
  - Velocity estimate exceeds the bounds once ($t = 64.5$ s) in 1000 timesteps.
  - That is, the error exceeds the bounds 0.1 % of the time, which is within the theoretical expectations of 99.7 % accuracy for the confidence bounds.



Summary

- You have now learned how to:
  - Implement the KF steps in Octave code.
  - Plot the output of the KF in two different ways to visualize results.
- The code is general—it is not specific to the spring-mass-damper system—so it can be used to estimate the state of any linear system obeying the KF assumptions (more on this in the next lessons, and in the next course).
- The sample results presented in this lesson give an intuitive sense for what to expect from KF.